

METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE

Documento de Arquitetura de Software

[Sigla] – [Nome do Projeto]

Gestor do Projeto	Gerente de Projeto
[nome]	[nome]
[email]	[email]
[telefone]	[telefone]

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
16/04/2026	-	Ana Carolina Gontijo Vilela Dias André Vinícius Guedes Martins Filipe Silva da Fonseca Gabriel César Peres Matos Gabriel Victor Vidal de Sales	Primeira versão deste documento	1.0

Sumário

1. INTRODUÇÃO	3
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
2. REPRESENTAÇÃO ARQUITETURAL	4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	5
4. VISÃO DE CASOS DE USO	5
4.1 Casos de Uso significantes para a arquitetura	5
5. VISÃO LÓGICA	7
5.1 Visão Geral – pacotes e camadas	7
6. VISÃO DE IMPLEMENTAÇÃO	9
6.1 Caso de Uso [00X]	9
6.1.1 Diagrama de Classes	9
6.1.2 Diagrama de Sequência	10
7. VISÃO DE IMPLANTAÇÃO	11
8. DIMENSIONAMENTO E PERFORMANCE	11
8.1 Volume	11
8.2 Performance	11
9. QUALIDADE	11

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema para Totens para atendimento em estabelecimentos alimentícios usando diversas visões de arquitetura para **representar** diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema. O documento irá adotar uma estrutura baseada na visão “4+1” de modelo de arquitetura [KRU41].

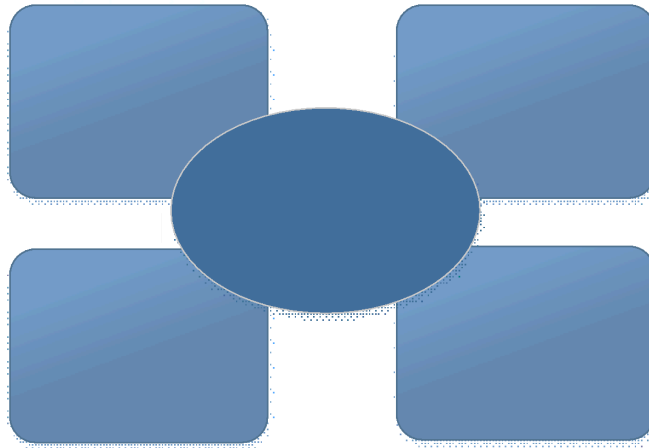


Figura 1 – Arquitetura 4+1

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao sistema para Totens para atendimento em estabelecimentos alimentícios, que será desenvolvido pelo grupo 3

1.3 Definições, Acrônimos e Abreviações

QoS – Quality of Service, ou qualidade de serviço. Termo utilizado para descrever um conjunto de qualidades que descrevem os requisitos não-funcionais de um sistema, como performance, disponibilidade e escalabilidade[QOS].

API REST - Interface de programação de aplicação baseadas em protocolo HTTP, seguindo a arquitetura REST (Representation State Transfer).

LGPD - Lei Geral de Proteção de Dados (Lei nº 13.709/2018).

UC - Caso de Uso (Use Case).

1.4 Referências

[KRU41]: The “4+1” view model of software architecture, Philippe Kruchten, November 1995, <http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>

[QOS] <https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html>

Documento de Visão do sistema, v1.0

https://docs.google.com/document/d/1mgDEouZhZa1B9pTFNFe62RDYVHrZyEgg/edit?usp=drive_link&oid=113031633604270678004&rtpof=true&sd=true

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseadas no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	Diagrama de Classe e Pacotes
Processo	Integradores	Performance, Escalabilidade, Concorrência	Diagrama de Sequência
Implementação	Programadores	Componentes de Software	Diagrama de Componentes
Implantação	Gerência de Configuração	Nodos físicos	Diagrama de Implantação
Caso de Uso	Todos	Requisitos funcionais	Diagrama de Caso de Uso
Dados	Especialistas em dados Administradores de dados	Persistência de dados	

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e restrições que tem um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	Back-end: Java 17+ com Spring Boot Front-end: JavaScript com React.js
Plataforma	Back-end executado em servidor com JVM (Java Virtual Machine). Front-end - servido por via build do React.js Sistema operacional - Linux, Windows
Segurança	Os dados de pagamento seguirão o padrão PCI-DSS Autenticação no painel administrativo via JWT (Json Web Token) Conformidade com a LGPD no tratamento de dados dos clientes.
Persistência	Banco de dados relacional PostgreSQL. Acesso JPA/Hibernate no back-end. No modo offline o totem armazena os pedidos localmente para sincronizar posteriormente.
Internacionalização (i18n)	O sistema desenvolvido em português do Brasil (pt-br).

Tabela 2 – Exemplo de requisitos e restrições

4. VISÃO DE CASOS DE USO

Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.

Lista de casos de uso do sistema:

Casos mais importantes:

- **UC1** – Processar pedido e pagamento
- **UC2** – Gerenciar pagamentos e relatórios
- **UC3** – Cliente realiza pedido
- **UC4** – Gerenciar cardápio

4.1 Casos de Uso significantes para a arquitetura

UC1 - Processar Pedido

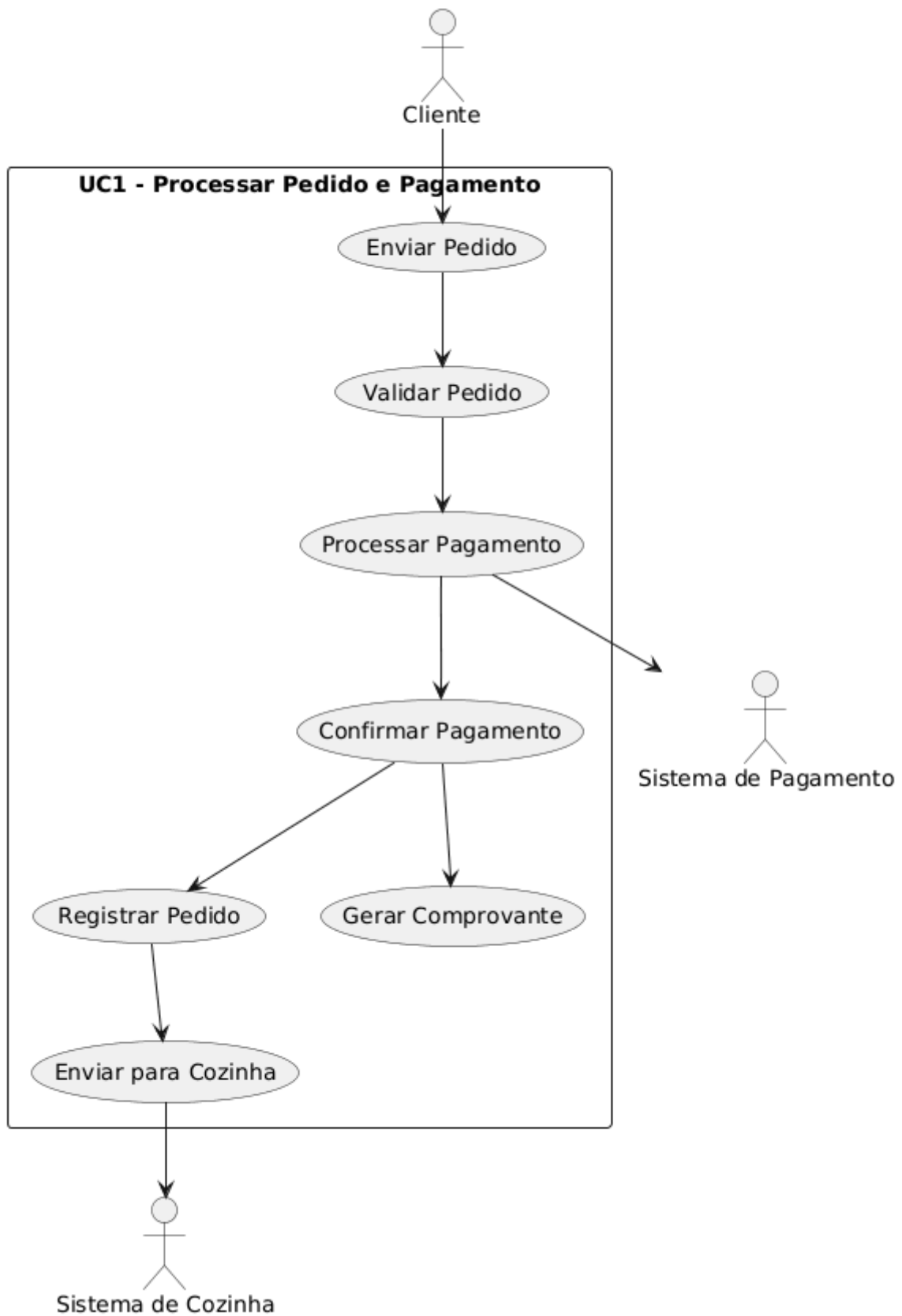
Impacta:

- Performance
- Segurança
- Integração com pagamentos

UC3 - Cliente realiza pedido no totem

Impacta:

- UX
- Tempo de resposta



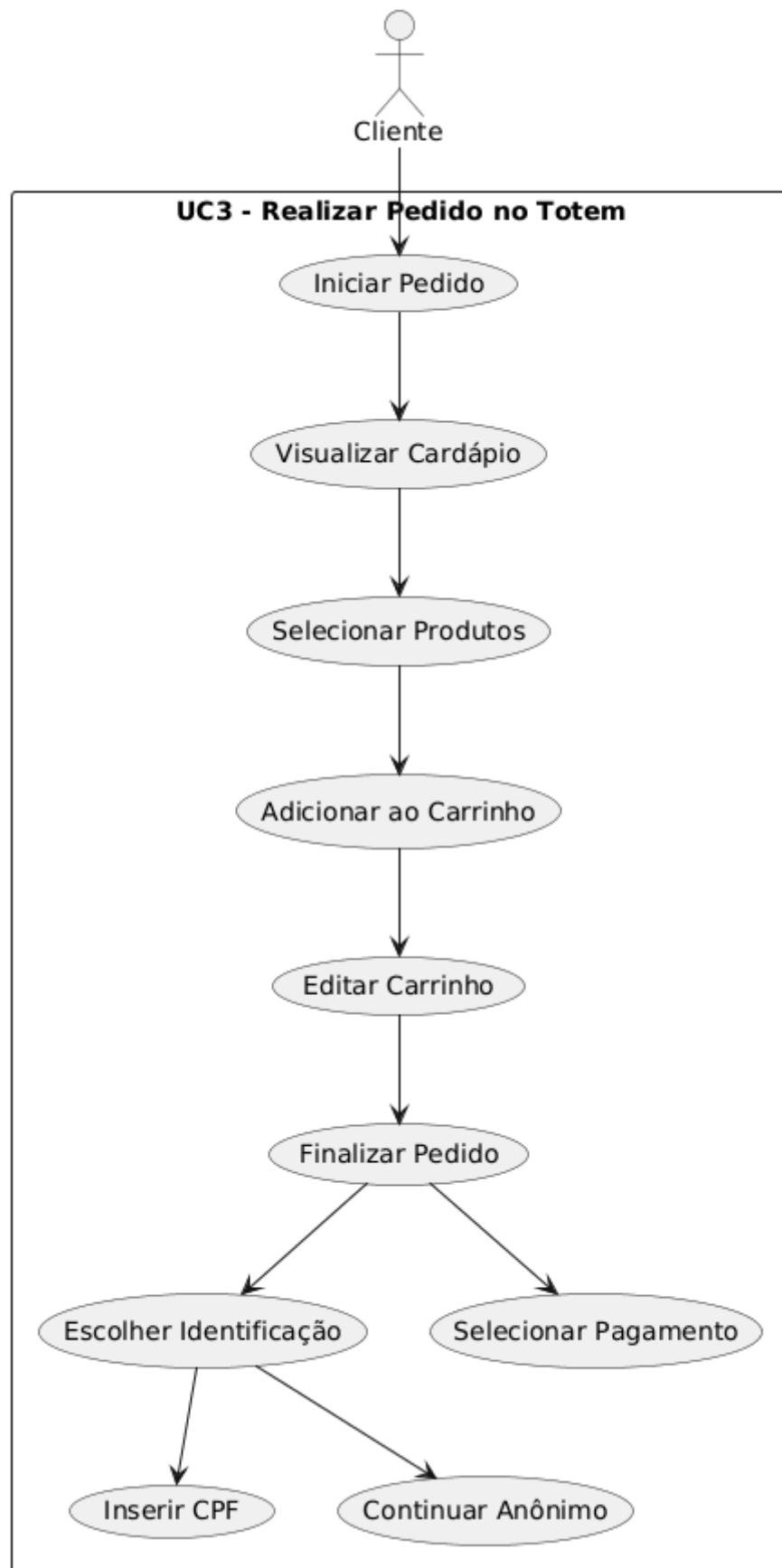


Figura 2 – Exemplo de Diagrama com os casos de uso significativos e atores

5. VISÃO LÓGICA

5.1 Visão Geral – pacotes e camadas

- Separação em camadas
- Front desacoplado do back
- Regras de negócio isoladas
- Integrações externas separadas

Apresentação

- Interface do totem (React)
- Painel admin (web)

Aplicação

- Controle de fluxo (controllers)

Domínio

- Serviços
- Regras de negócio (pedido, pagamento)
- Entidades do sistema

Infraestrutura

- Banco de dados
- APIs externas

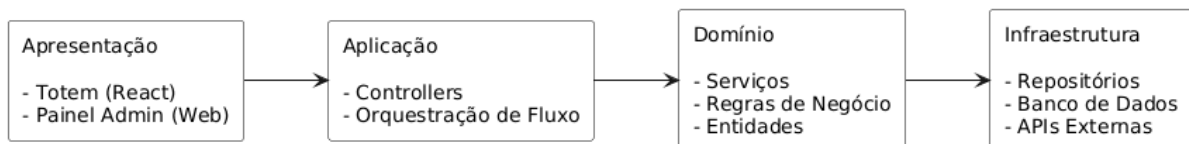


Figura 3 – Exemplo de Diagrama de Camadas da Aplicação

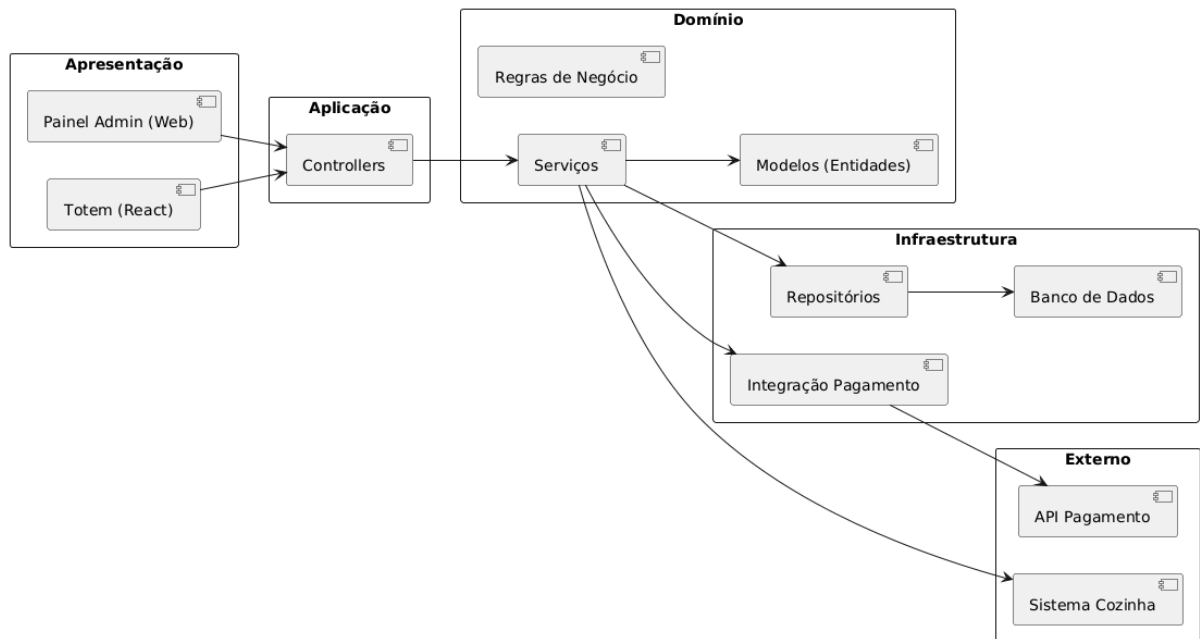


Figura 4 – Exemplo de Diagrama de Pacotes da Aplicação

6. VISÃO DE IMPLEMENTAÇÃO

6.1 Caso de Uso: UC1 – Processar Pedido

6.1.1 Diagrama de Classes

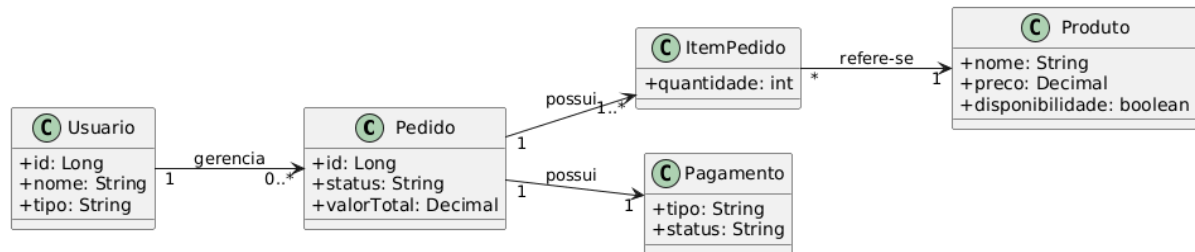


Figura 5 – Exemplo de Diagrama de Pacotes da Aplicação

6.1.2 Diagrama de Sequência

O diagrama de sequência representa o fluxo completo de processamento de um pedido, desde a interação do cliente no totem até a confirmação do pagamento e envio para a cozinha.

Fluxo:

1. Cliente seleciona itens
2. Front envia requisição → API
3. API cria pedido
4. Integra com pagamento
5. Pagamento aprovado
6. Pedido salvo no banco
7. Enviado para cozinha
8. Retorna confirmação

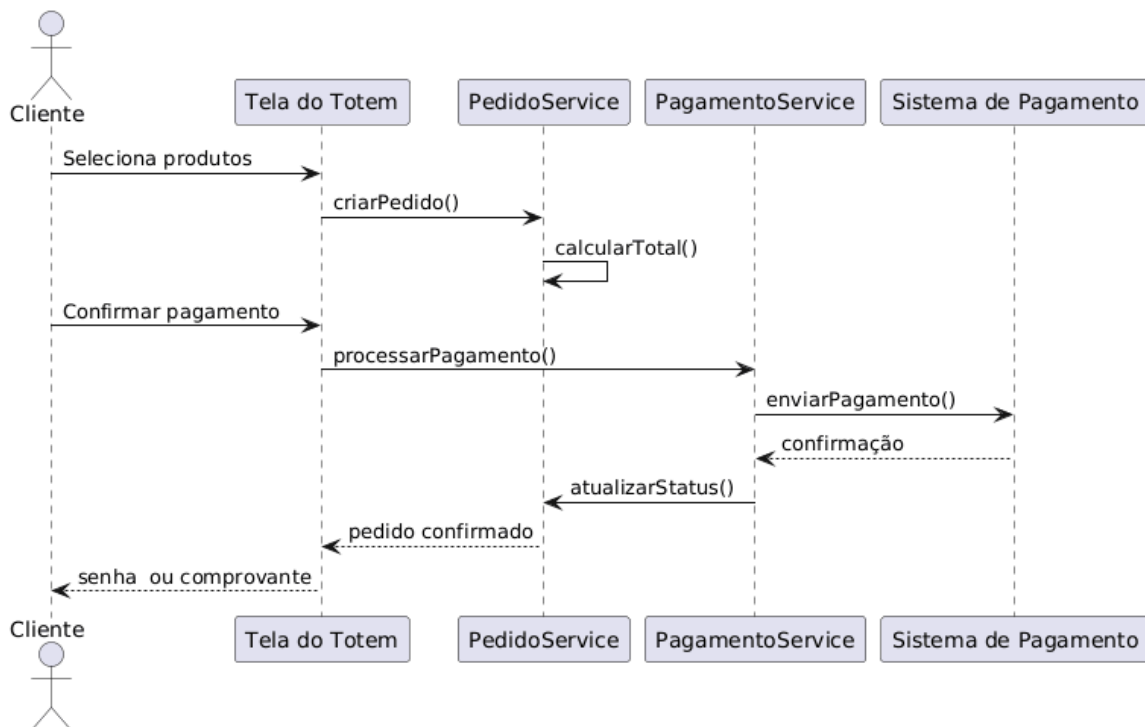


Figura 6 – Exemplo de Diagrama de Sequência

6.2 Caso de Uso: UC4 – Gerenciar Cardápio

6.2.1 Diagramas de Classes

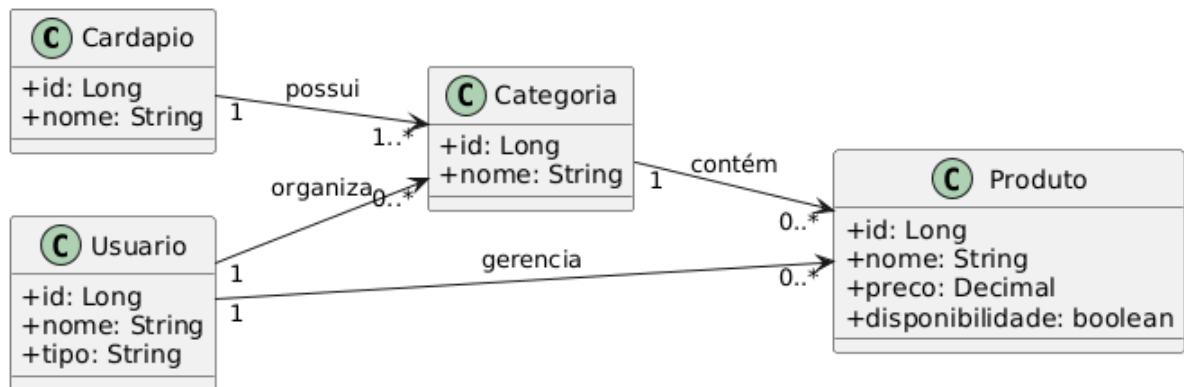


Figura 7– O diagrama de classes apresenta as principais entidades relacionadas ao gerenciamento do cardápio, incluindo Cardápio, Categoria, Produto e Usuário, bem como seus relacionamentos e responsabilidades dentro do sistema.

6.3 Diagrama de Componentes

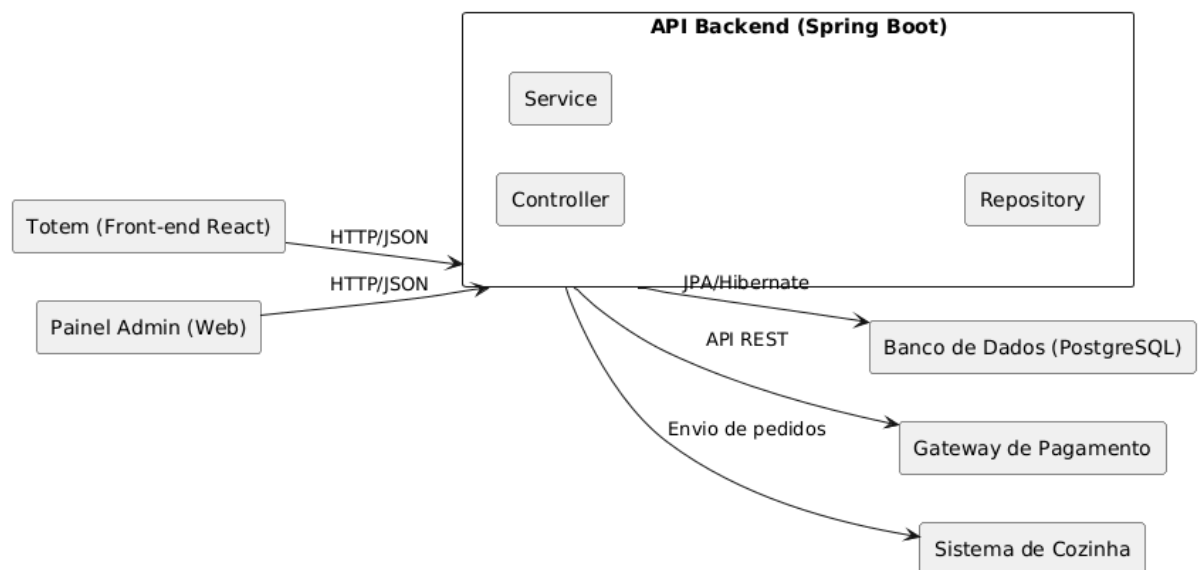


Figura 8– Exemplo de Diagrama de Componentes

7. VISÃO DE IMPLANTAÇÃO

A arquitetura de implantação do sistema é baseada em um modelo cliente-servidor distribuído, composto por três principais nós físicos:

Nós Físicos

Totem de Autoatendimento (Cliente)

- Hardware dedicado com touchscreen
- Sistema operacional: Linux ou Windows
- Executa aplicação front-end (React.js) em modo kiosk
- Possui armazenamento local para operação offline (cache de pedidos)

Servidor de Aplicação (Back-end)

- Ambiente com JVM (Java Virtual Machine)
- Executa aplicação Spring Boot
- Responsável por:
 - Processamento de pedidos
 - Regras de negócio
 - Autenticação (JWT)
 - Integração com pagamentos

Servidor de Banco de Dados

- PostgreSQL
- Armazenamento persistente dos dados:
 - Pedidos
 - Usuários
 - Produtos
 - Transações

Comunicação

- Comunicação via HTTP/HTTPS utilizando API REST
- JSON como formato de troca de dados
- Sincronização offline:
 - Totem armazena pedidos localmente
 - Envio automático ao servidor quando conexão for restabelecida

Considerações

- Uso de HTTPS para garantir segurança dos dados
- Possibilidade de implantação em nuvem (AWS, Azure, etc.)
- Suporte a múltiplos totens conectados ao mesmo servidor

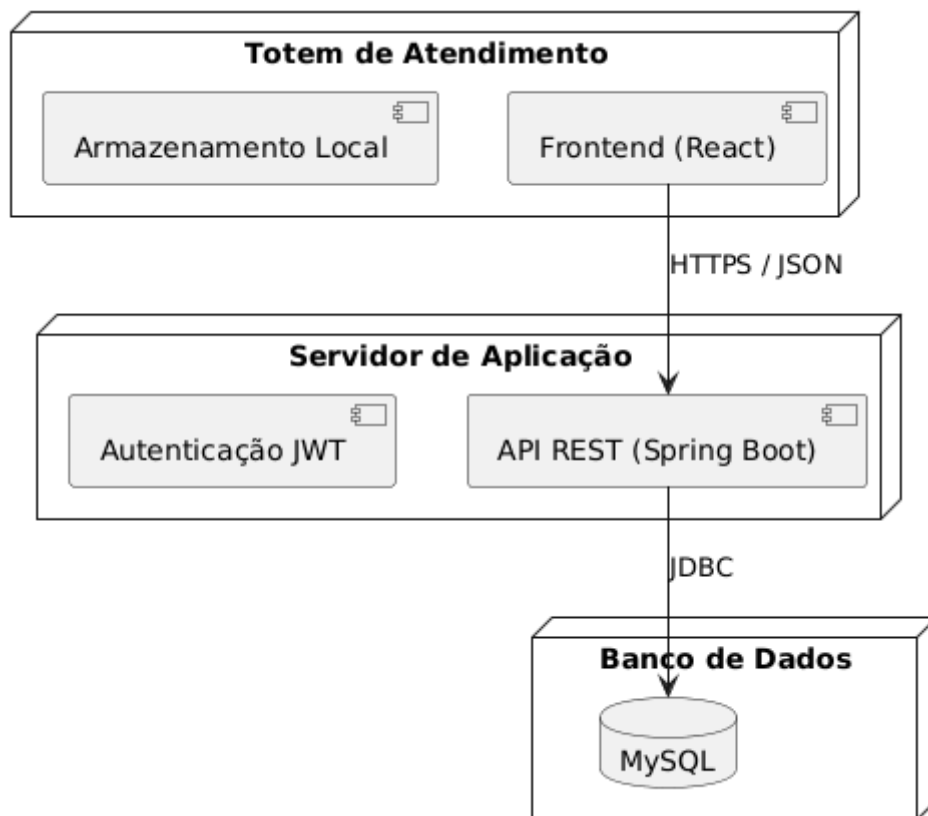


Figura 9– Exemplo de Diagrama de Implantação

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Número de estimado usuários: 300 usuários/dia por estabelecimento
Número estimado de acessos diários: 1.200 requisições/dia
Número estimado de acessos por período (pico): 100 requisições/hora
Tempo de sessão de um usuário: 3 a 7 minutos

8.2 Performance

Tempo máximo para a execução de determinada transação:

- Cadastro de pedido: até 2 segundos
- Consulta de cardápio: até 1 segundo
- Finalização de pagamento: até 3 segundos

Estratégias de Performance

- Cache de cardápio no cliente (totem)
- Uso de conexão persistente HTTP
- Indexação no banco de dados (PostgreSQL)
- Processamento assíncrono para envio de pedidos offline

- Escalabilidade horizontal do back-end (load balancing)

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	Capacidade de suportar aumento de usuários e totens	Arquitetura baseada em serviços com possibilidade de escalabilidade horizontal (replicação do back-end e balanceamento de carga)
Confiabilidade, Disponibilidade	Sistema deve permanecer operacional mesmo com falhas de conexão	Modo offline no totem com sincronização posterior; retries automáticos; logs de falha
Portabilidade	Sistema deve funcionar em diferentes ambientes	Uso de Java (JVM) no back-end e aplicação web no front-end (React), compatível com múltiplos sistemas operacionais
Segurança	Proteção de dados sensíveis e transações	HTTPS, autenticação via JWT, conformidade com LGPD, práticas PCI-DSS para pagamentos